

O que é o GraphQL?

GraphQL é uma **linguagem de consulta para APIs**.

Ele permite que o **cliente (frontend)** diga **exatamente** quais dados quer receber do **servidor (backend)** — nem mais, nem menos.

👉 Pensa nele como um **cardápio inteligente**:

Você pede **exatamente o que quer comer**, e o garçom traz só isso — em vez de te trazer o prato completo com coisas que você não pediu.

GraphQL vs REST (comparação simples)

Conceito	REST	GraphQL
Endpoints	Muitos (<code>/users</code> , <code>/users/1/posts</code>)	Um só (<code>/graphql</code>)
Verbos HTTP	GET, POST, PUT, DELETE	Normalmente só POST
Formato da resposta	Fixo pelo servidor	Definido pelo cliente
Problemas comuns	Overfetching (dados demais) e Underfetching (dados de menos)	Nenhum dos dois — o cliente escolhe

Estrutura básica de uma API GraphQL

Uma API GraphQL é formada por 3 partes principais:

- 1 **Schema** — define a *forma dos dados* e o que pode ser consultado ou alterado.
- 2 **Query** — usada para *consultar* dados (equivalente ao GET do REST).
- 3 **Mutation** — usada para *alterar* dados (criar, editar, excluir).

1. Schema

O **schema** é o "contrato" entre cliente e servidor.

Ele define **tipos**, **campos** e **operações** disponíveis.

Exemplo:

```
type User {
```

```
  id: ID!
  name: String!
  email: String!
}

type Query {
  users: [User]
  user(id: ID!): User
}

type Mutation {
  createUser(name: String!, email: String!): User
}
```

 Explicando:

- **User**: tipo de dado (como uma “tabela” ou “classe”).
 - **Query**: define o que podemos *buscar*.
 - **Mutation**: define o que podemos *modificar*.
 - **!**: indica que o campo é obrigatório.
-

2. Query (consultas)

Uma **query** é como uma requisição de leitura.

Exemplo:

```
{
  user(id: 1) {
    name
    email
  }
}
```

 Interpretação:

"Me traga o usuário com **id=1**, mas só os campos **name** e **email**."

Resposta do servidor:

```
{
  "data": {
    "user": {
      "name": "Vilson",
      "email": "vilson@email.com"
    }
  }
}
```

📌 O servidor retorna **somente** o que foi pedido!

3. Mutation (alterações)

As **mutations** permitem *criar, atualizar ou apagar* dados.

Exemplo:

```
mutation {
  createUser(name: "Maria", email: "maria@email.com") {
    id
    name
  }
}
```

Resposta:

```
{
  "data": {
    "createUser": {
      "id": 3,
      "name": "Maria"
    }
  }
}
```

4. Resolver

Os **resolvers** são as *funções que respondem às queries e mutations*.

No backend, você cria algo assim:

```
const root = {
  users: () => bancoDeUsuarios,
  user: ({ id }) => bancoDeUsuarios.find(u => u.id == id),
  createUser: ({ name, email }) => { ... }
};
```

O GraphQL chama essas funções de acordo com o que o cliente pediu.

5. Vantagens do GraphQL

Vantagem	Explicação
 Precisão	O cliente recebe só o que pediu
 Menos requisições	Pode obter vários dados em uma única chamada
 Evolutivo	Adicionar novos campos sem quebrar clientes antigos
 Tipagem forte	O schema define os tipos, ajudando no desenvolvimento
 Ferramentas	Interfaces como GraphiQL e Apollo facilitam testes

6. Como funciona na prática

1 O cliente envia uma **query** (texto em JSON) para `/graphql`

2 O servidor GraphQL:

- Lê o schema
- Identifica os resolvers necessários
- Executa as funções
- Retorna só os campos pedidos

💬 Tudo é baseado em **declarações**, não em rotas.

Exemplo real de consulta

```
{
  user(id: 2) {
    name
    email
    posts {
      title
      comments {
        text
      }
    }
  }
}
```

💡 Aqui o cliente faz **uma única requisição**, mas obtém:

- dados do usuário,
- seus posts,
- e os comentários dos posts.

Em REST, isso exigiria **várias requisições** separadas.

7. Ferramentas populares

Categoria	Ferramentas
Servidor (backend)	Apollo Server, Express-GraphQL, Yoga
Cliente (frontend)	Apollo Client, Relay, urql
Geradores automáticos	Hasura, PostGraphile
IDE de testes	GraphiQL, GraphQL Playground

Resumo final

Conceito	Significado
----------	-------------

Schema	Define estrutura e tipos dos dados
Query	Busca dados
Mutation	Altera dados
Resolver	Funções que executam cada operação
Endpoint único	<code>/graphql</code>
Principal vantagem	Cliente controla exatamente o que recebe